

TEORETICKÁ INFORMATIKA

Úvod do algoritmicky těžkých problémů

David Weber
weber3@spsejecna.cz



Rok 2025/26

Těžké problémy

S čím jsme se již setkali. . .

S čím jsme se již setkali. . .

- Řazení prvků v poli $\implies O(n \log n)$.

S čím jsme se již setkali. . .

- Řazení prvků v poli $\implies O(n \log n)$.
- Hledání nejkratší cesty v neohodnoceném grafu $\implies O(n + m)$,
přičemž $m \leq \frac{n(n-1)}{2}$

S čím jsme se již setkali. . .

- Řazení prvků v poli $\implies O(n \log n)$.
- Hledání nejkratší cesty v neohodnoceném grafu $\implies O(n + m)$,
přičemž $m \leq \frac{n(n-1)}{2}$
- Hledání nejkratší cesty v nezáporně ohodnoceném grafu
 $\implies O((n + m) \log n)$.

S čím jsme se již setkali. . .

- Řazení prvků v poli $\implies O(n \log n)$.
- Hledání nejkratší cesty v neohodnoceném grafu $\implies O(n + m)$,
přičemž $m \leq \frac{n(n-1)}{2}$
- Hledání nejkratší cesty v nezáporně ohodnoceném grafu
 $\implies O((n + m) \log n)$.
- Sekvenční vyhledávání v poli $\implies O(n)$.

S čím jsme se již setkali...

- Řazení prvků v poli $\implies O(n \log n)$.
- Hledání nejkratší cesty v neohodnoceném grafu $\implies O(n + m)$,
přičemž $m \leq \frac{n(n-1)}{2}$
- Hledání nejkratší cesty v nezáporně ohodnoceném grafu
 $\implies O((n + m) \log n)$.
- Sekvenční vyhledávání v poli $\implies O(n)$.
- Binární vyhledávání v poli $\implies O(\log n)$.

S čím jsme se již setkali...

- Řazení prvků v poli $\implies O(n \log n)$.
- Hledání nejkratší cesty v neohodnoceném grafu $\implies O(n + m)$,
přičemž $m \leq \frac{n(n-1)}{2}$
- Hledání nejkratší cesty v nezáporně ohodnoceném grafu
 $\implies O((n + m) \log n)$.
- Sekvenční vyhledávání v poli $\implies O(n)$.
- Binární vyhledávání v poli $\implies O(\log n)$.
- Vyhledávání v textu $\implies O((n - m)m)$, kde $m \leq n$ je délka
hledaného slova.

S čím jsme se již setkali...

- Řazení prvků v poli $\implies O(n \log n)$.
- Hledání nejkratší cesty v neohodnoceném grafu $\implies O(n + m)$,
přičemž $m \leq \frac{n(n-1)}{2}$
- Hledání nejkratší cesty v nezáporně ohodnoceném grafu
 $\implies O((n + m) \log n)$.
- Sekvenční vyhledávání v poli $\implies O(n)$.
- Binární vyhledávání v poli $\implies O(\log n)$.
- Vyhledávání v textu $\implies O((n - m)m)$, kde $m \leq n$ je délka
hledaného slova.

Všechny tyto problémy umíme řešit obecně v čase $O(n^k)$,
kde $k \in \mathbb{R}$.

- **Binární kódování** vstupu x budeme značit $\langle x \rangle_B$.

Formální pojetí problémů

- Omezíme se na tzv. **rozhodovací problémy**.

- **Binární kódování** vstupu x budeme značit $\langle x \rangle_B$.

Formální pojetí problémů

- Omezíme se na tzv. **rozhodovací problémy**.

Rozhodovací problém je takový problém, na jehož výstupní hodnoty jsou **ANO** a **NE**.

- **Binární kódování** vstupu x budeme značit $\langle x \rangle_B$.

Formální pojetí problémů

- Omezíme se na tzv. **rozhodovací problémy**.

Rozhodovací problém je takový problém, na jehož výstupní hodnoty jsou **ANO** a **NE**.

- Označme si $\{0, 1\}^*$ **všechny konečné posloupnosti nul a jedniček**.

- **Binární kódování** vstupu x budeme značit $\langle x \rangle_B$.

Formální pojetí problémů

- Omezíme se na tzv. **rozhodovací problémy**.

Rozhodovací problém je takový problém, na jehož výstupní hodnoty jsou **ANO** a **NE**.

- Označme si $\{0, 1\}^*$ **všechny konečné posloupnosti nul a jedniček**.
- Rozhodovacím problémem formálně rozumíme zobrazení

$$L : \{0, 1\}^* \rightarrow \{0, 1\}.$$

- **Binární kódování** vstupu x budeme značit $\langle x \rangle_B$.

Formální pojetí problémů

- Omezíme se na tzv. **rozhodovací problémy**.

Rozhodovací problém je takový problém, na jehož výstupní hodnoty jsou **ANO** a **NE**.

- Označme si $\{0, 1\}^*$ **všechny konečné posloupnosti nul a jedniček**.
- Rozhodovacím problémem formálně rozumíme zobrazení

$$L : \{0, 1\}^* \rightarrow \{0, 1\}.$$

Tzn. vstupní data kódujeme **binárně**, tj. pomocí nul a jedniček.

- **Binární kódování** vstupu x budeme značit $\langle x \rangle_B$.

Efektivně řešitelné problémy

Efektivně řešitelné problémy

S jistou rezervou lze říci, že problémy, které lze řešit algoritmem, který běží v polynomiálním čase vzhledem k délce vstupu, jsou
„efektivně řešitelné“.

Efektivně řešitelné problémy

S jistou rezervou lze říci, že problémy, které lze řešit algoritmem, který běží v polynomiálním čase vzhledem k délce vstupu, jsou
„efektivně řešitelné“.

Co představuje zmíněnou rezervu?

Efektivně řešitelné problémy

S jistou rezervou lze říci, že problémy, které lze řešit algoritmem, který běží v polynomiálním čase vzhledem k délce vstupu, jsou **„efektivně řešitelné“**.

Co představuje zmíněnou rezervu?

- Algoritmus běžící v čase $O(n^{1000})$ je polynomiální, ale v praxi nejspíše jen sotva použitelný.

Efektivně řešitelné problémy

S jistou rezervou lze říci, že problémy, které lze řešit algoritmem, který běží v polynomiálním čase vzhledem k délce vstupu, jsou **„efektivně řešitelné“**.

Co představuje zmíněnou rezervu?

- Algoritmus běžící v čase $O(n^{1000})$ je polynomiální, ale v praxi nejspíše jen sotva použitelný.
- Naopak algoritmus se složitostí $O(0,001^n)$ je sice exponenciální, ale je jistě použitelný na „rozumná“ data.

Efektivně řešitelné problémy

S jistou rezervou lze říci, že problémy, které lze řešit algoritmem, který běží v polynomiálním čase vzhledem k délce vstupu, jsou **„efektivně řešitelné“**.

Co představuje zmíněnou rezervu?

- Algoritmus běžící v čase $O(n^{1000})$ je polynomiální, ale v praxi nejspíše jen sotva použitelný.
- Naopak algoritmus se složitostí $O(0,001^n)$ je sice exponenciální, ale je jistě použitelný na „rozumná“ data.

Takové případy ale můžeme zanedbat, neboť jsou **velmi řídké**.

Proč zrovna polynomy?

Proč zrovna polynomy?

- 1 Polynomy jsou tzv. **uzavřeny** na sčítání, násobení a skládání. Mějme polynomy $p_1(x)$ a $p_2(y)$. Pak
- $p_1(x) + p_2(x)$ je polynom
 - $p_1(x) \cdot p_2(x)$ je polynom
 - $p_1(p_2(x))$ je polynom

Proč zrovna polynomy?

- 1 Polynomy jsou tzv. **uzavřeny** na sčítání, násobení a skládání. Mějme polynomy $p_1(x)$ a $p_2(y)$. Pak
 - $p_1(x) + p_2(x)$ je polynom
 - $p_1(x) \cdot p_2(x)$ je polynom
 - $p_1(p_2(x))$ je polynom
- 2 Polynomy obvykle nerostou „příliš rychle“.

Proč zrovna polynomy?

Příklad

Proč zrovna polynomy?

Příklad

$$p_1(x) = x^2 - x - 6$$

$$p_2(x) = 3x^2 + 5$$

Proč zrovna polynomy?

Příklad

$$p_1(x) = x^2 - x - 6$$

$$p_2(x) = 3x^2 + 5$$

- $p_1(x) + p_2(x) = (x^2 - x - 6) + (3x^2 + 5) = 4x^2 - x - 1$

Proč zrovna polynomy?

Příklad

$$p_1(x) = x^2 - x - 6$$

$$p_2(x) = 3x^2 + 5$$

- $p_1(x) + p_2(x) = (x^2 - x - 6) + (3x^2 + 5) = 4x^2 - x - 1$
- $p_1(x) \cdot p_2(x) = (x^2 - x - 6) \cdot (3x^2 + 5) = 3x^4 - 3x^3 - 13x^2 - 5x - 30$

Proč zrovna polynomy?

Příklad

$$p_1(x) = x^2 - x - 6$$

$$p_2(x) = 3x^2 + 5$$

- $p_1(x) + p_2(x) = (x^2 - x - 6) + (3x^2 + 5) = 4x^2 - x - 1$
- $p_1(x) \cdot p_2(x) = (x^2 - x - 6) \cdot (3x^2 + 5) = 3x^4 - 3x^3 - 13x^2 - 5x - 30$
- $p_1(p_2(x)) = ((3x^2 + 5)^2 - (3x^2 + 5) - 6) = 9x^4 + 27x^2 + 14$

Proč zrovna polynomy?

Příklad

$$p_1(x) = x^2 - x - 6$$

$$p_2(x) = 3x^2 + 5$$

- $p_1(x) + p_2(x) = (x^2 - x - 6) + (3x^2 + 5) = 4x^2 - x - 1$
- $p_1(x) \cdot p_2(x) = (x^2 - x - 6) \cdot (3x^2 + 5) = 3x^4 - 3x^3 - 13x^2 - 5x - 30$
- $p_1(p_2(x)) = ((3x^2 + 5)^2 - (3x^2 + 5) - 6) = 9x^4 + 27x^2 + 14$

Výsledkem je zase polynom.

Oecnějšší pohled

Je každý problém řešitelný polynomiálním algoritmem?

Je každý problém řešitelný polynomiálním algoritmem?

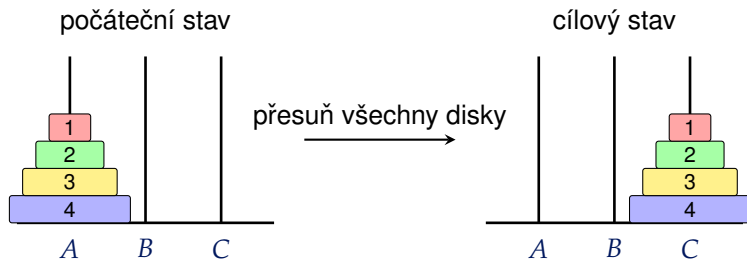
- **Nikoliv!** Existují dokonce problémy, které nejsou řešitelné žádným algoritmem.

Je každý problém řešitelný polynomiálním algoritmem?

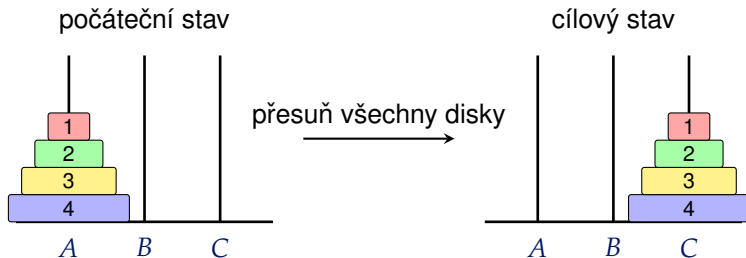
- **Nikoliv!** Existují dokonce problémy, které nejsou řešitelné žádným algoritmem.
- Např. Hanojské věže nelze řešit lépe než exponenciálním algoritmem.

Hanojské věže

Hanojské věže

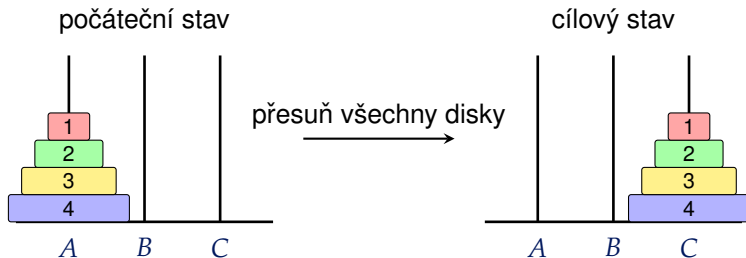


Hanojské věže



Pravidla:

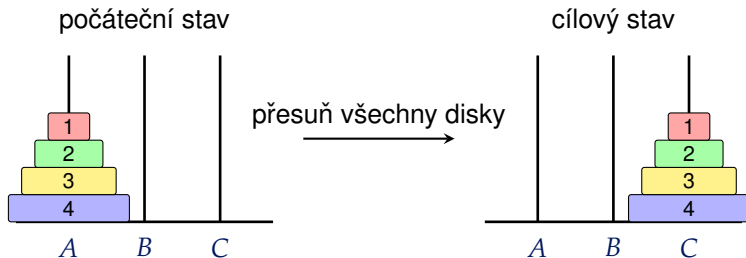
Hanojské věže



Pravidla:

- V každém kroku můžeme přesunout jen jeden disk.

Hanojské věže



Pravidla:

- V každém kroku můžeme přesunout jen jeden disk.
- Větší disk nesmí ležet na menším.

Algoritmus pro Hanojské věže

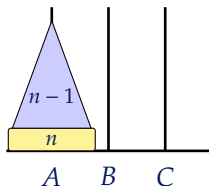
Algoritmus: HANOJ

Vstup: Výška věže n ; sloupy A (zdrojový), B (cílový), C (pomocný)

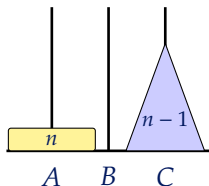
```
1 if  $n = 1$  then přesuň disk 1 z  $A$  na  $B$ ;  
2 else  
3   Zavolej  $\text{HANOJ}(n - 1, A, C, B)$   
4   Přesuneme disk  $n$  z  $A$  na  $B$   
5   Zavolej  $\text{HANOJ}(n - 1, C, B, A)$ 
```

Znázornění algoritmu

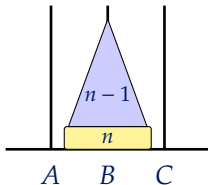
přesuň prvních $n - 1$ disků
z A na C



přesuň disk n
z A na B



přesuň prvních $n - 1$ disků
z C na B



Časová složitost algoritmu

Časová složitost algoritmu

- Označme si počet kroků algoritmu $T(n)$.

Časová složitost algoritmu

- Označme si počet kroků algoritmu $T(n)$.
- Zjevně pro $n = 1$ stačí jen přesunout disk z počátečního na cílový kolík, tzn. $T(1) = 1$.

Časová složitost algoritmu

- Označme si počet kroků algoritmu $T(n)$.
- Zjevně pro $n = 1$ stačí jen přesunout disk z počátečního na cílový kolík, tzn. $T(1) = 1$.
- Jinak rekurzivně voláme dvakrát $T(n)$ a navíc provedeme přesun největšího disku.

$$T(n) = 2 \cdot T(n - 1) + 1.$$

Časová složitost algoritmu

- Označme si počet kroků algoritmu $T(n)$.
- Zjevně pro $n = 1$ stačí jen přesunout disk z počátečního na cílový kolík, tzn. $T(1) = 1$.
- Jinak rekurzivně voláme dvakrát $T(n)$ a navíc provedeme přesun největšího disku.

$$T(n) = 2 \cdot T(n - 1) + 1.$$

n	1	2	3	4	5	...
$T(n)$	1	3	7	15	31	...

Časová složitost algoritmu

- Označme si počet kroků algoritmu $T(n)$.
- Zjevně pro $n = 1$ stačí jen přesunout disk z počátečního na cílový kolík, tzn. $T(1) = 1$.
- Jinak rekurzivně voláme dvakrát $T(n)$ a navíc provedeme přesun největšího disku.

$$T(n) = 2 \cdot T(n - 1) + 1.$$

n	1	2	3	4	5	...
$T(n)$	1	3	7	15	31	...

- Lze odvodit explicitní vzorec $T(n) = 2^n - 1$.

Časová složitost algoritmu

- Označme si počet kroků algoritmu $T(n)$.
- Zjevně pro $n = 1$ stačí jen přesunout disk z počátečního na cílový kolík, tzn. $T(1) = 1$.
- Jinak rekurzivně voláme dvakrát $T(n)$ a navíc provedeme přesun největšího disku.

$$T(n) = 2 \cdot T(n - 1) + 1.$$

n	1	2	3	4	5	...
$T(n)$	1	3	7	15	31	...

- Lze odvodit explicitní vzorec $T(n) = 2^n - 1$.
- Algoritmus je tedy exponenciální, tj. $O(2^n)$.

Časová složitost algoritmu

- Označme si počet kroků algoritmu $T(n)$.
- Zjevně pro $n = 1$ stačí jen přesunout disk z počátečního na cílový kolík, tzn. $T(1) = 1$.
- Jinak rekurzivně voláme dvakrát $T(n)$ a navíc provedeme přesun největšího disku.

$$T(n) = 2 \cdot T(n - 1) + 1.$$

n	1	2	3	4	5	...
$T(n)$	1	3	7	15	31	...

- Lze odvodit explicitní vzorec $T(n) = 2^n - 1$.
- Algoritmus je tedy exponenciální, tj. $O(2^n)$.

Lépe to ani nelze provést!

Rozhodovací Hanojské věže

Hanojské věže ale nepředstavují rozhodovací problém
(nikde není výstupem **ANO** nebo **NE**).

Hanojské věže ale nepředstavují rozhodovací problém (nikde není výstupem **ANO** nebo **NE**).

- Zkusíme mírně změnit zadání: Ke vstupu přidáme ještě číslo $k \in \mathbb{N}$.

Hanojské věže ale nepředstavují rozhodovací problém (nikde není výstupem **ANO** nebo **NE**).

- Zkusíme mírně změnit zadání: Ke vstupu přidáme ještě číslo $k \in \mathbb{N}$.
- **Otázka:** Lze přemístit celou věž ze sloupu A na sloup B v nejvýše k tazích?

Hanojské věže ale nepředstavují rozhodovací problém (nikde není výstupem **ANO** nebo **NE**).

- Zkusíme mírně změnit zadání: Ke vstupu přidáme ještě číslo $k \in \mathbb{N}$.
- **Otázka:** Lze přemístit celou věž ze sloupu A na sloup B v nejvýše k tazích?
- K rozhodnutí odpovědi použijeme upravený původní algoritmus.

Problém zastavení

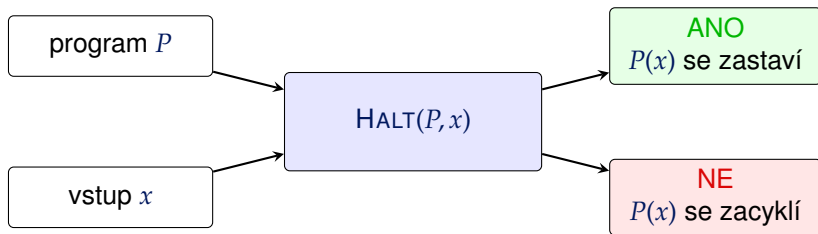
- Máme na vstupu program P a jeho vstup x .

Problém zastavení

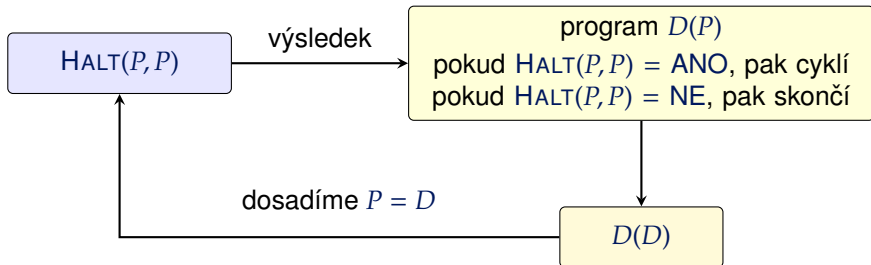
- Máme na vstupu program P a jeho vstup x .
- Uvažujme problém $\text{HALT}(P, x)$, který rozhodne, zdali se program P nad vstupem x **zastaví**, a nebo se **zacyklí**.

Problém zastavení

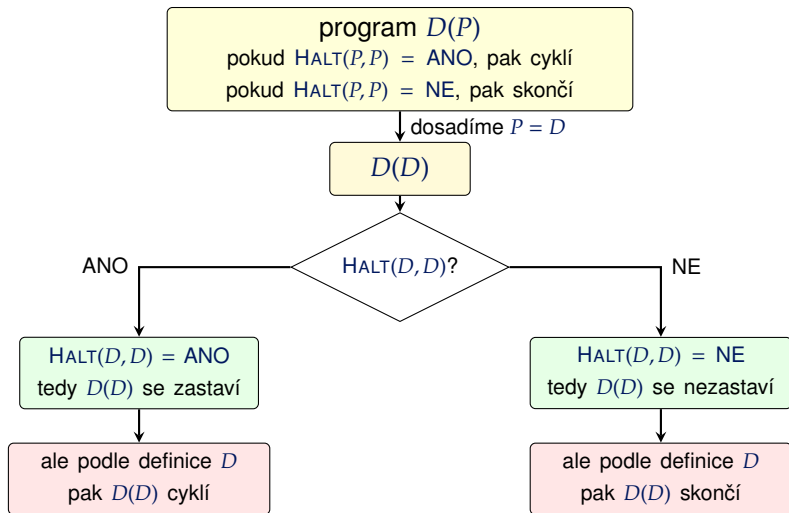
- Máme na vstupu program P a jeho vstup x .
- Uvažujme problém $\text{HALT}(P, x)$, který rozhodne, zdali se program P nad vstupem x **zastaví**, a nebo se **zacyklí**.



Problém **HALT** nelze rozhodnout



Problém **HALT** nelze rozhodnout



V obou případech vznikne spor $\Rightarrow$ Algoritmus pro **HALT** neexistuje.

Co z toho plyne?

Co z toho plyne?

- Existence „rychlého“ (polynomiálního) řešení problému není vždy samozřejmostí.

Co z toho plyne?

- Existence „rychlého“ (polynomiálního) řešení problému není vždy samozřejmostí.
- Některé problémy dokonce nelze řešit vůbec!

Co z toho plyne?

- Existence „rychlého“ (polynomiálního) řešení problému není vždy samozřejmostí.
- Některé problémy dokonce nelze řešit vůbec!

Mezi různými problémy však lze nalézt zajímavé vztahy.

- PAPADIMITRIOU, Christos H. *Computational complexity*. Reading: Addison-Wesley, 1994. ISBN 0-201-53082-1.
- MAREŠ, Martin a VALLA, Tomáš. *Průvodce labyrintem algoritmů*. Druhé vydání. CZ.NIC. Praha: CZ.NIC, z.s.p.o., 2022. ISBN 978-80-88168-63-8.